

Shruti in plain words

Every technical term used to build the app, what it means, what it is in Shruti, and the sentence to say out loud when a judge asks.

THE PRODUCT IN ONE BREATH

“In an Indian OPD the patient speaks Hindi but the record must be in English. Shruti listens in the patient’s language, drafts a structured case sheet, marks every AI-written word, and lets the doctor accept, edit or reject each one before signing. It documents – it never diagnoses.”

SECTION 1

How the app is built

The words that come up when a judge asks “what did you build this with?”

Frontend

The part of the app that runs in the browser — what the user sees and clicks.

IN SHRUTI • The dashboard, patient list, consultation workspace and review page.

Backend

Code that runs on a server, not in the browser. It holds secrets and talks to the database and the AI.

IN SHRUTI • Three route handlers under `/api/ai/*` and the auth gate.

SAY • *Our backend is small on purpose: three AI endpoints and one auth check.*

Next.js

A framework that lets one project contain both frontend and backend, in one language.

IN SHRUTI • The whole app is one Next.js project.

SAY • *One codebase, one deploy – there is no separate server to keep in sync.*

React

The library Next.js uses to build screens out of reusable pieces.

IN SHRUTI • Every screen is a React component.

TypeScript

JavaScript with types: the computer checks that data has the right shape before the code runs.

IN SHRUTI · Front and back. One file, `lib/types.ts`, defines what a Patient or a Consultation is, and both sides use it.

SAY · *Frontend and backend share one type definition, so a mismatch is caught at compile time – not on stage.*

Component

A self-contained, reusable piece of UI — a button, a card, a whole panel.

IN SHRUTI · `SectionEditor`, `VitalsForm`, `GapsPanel`, `NoteDocument`.

Tailwind CSS

Styling by adding small utility classes directly in the markup, instead of separate stylesheets.

IN SHRUTI · All styling.

shadcn/ui

Ready-made, accessible components (buttons, dialogs, tabs) that you copy into your project and own.

IN SHRUTI · Buttons, inputs, tabs, dialogs, cards.

SAY · *We didn't build a component library; we used an accessible one and styled it.*

Route handler

API endpoint

A URL on the server that runs code when it is called.

IN SHRUTI · `POST /api/ai/structure`, `/api/ai/gaps`, `/api/ai/summary`.

SAY · *The browser never talks to the AI directly – it calls our route handler, which holds the key.*

Proxy

middleware

Code that runs before every request and can redirect or block it.

IN SHRUTI · `proxy.ts` sends you to `/login` if you're not signed in and returns 401 on API calls without a session.

SAY · *Every request passes an auth check before it reaches a page.*

Client vs server component

Some components render on the server, some in the browser. Anything interactive — typing, buttons, the microphone — must run in the browser.

IN SHRUTI · Almost every screen is a client component because the workspace is interactive.

Environment variable

A setting stored outside the code — usually a secret or a switch — read when the app starts.

IN SHRUTI · `.env.local` locally and Vercel settings in production: the Supabase URL and key, the Gemini key, `NEXT_PUBLIC_AI_MODE` .

SAY · *Secrets live in environment variables, never in the repository.*

NEXT_PUBLIC_ prefix

Marks an environment variable as safe to send to the browser. Anything without it stays on the server.

IN SHRUTI · The Supabase URL and key are public; the Gemini key is not.

SAY · *The AI key never reaches the browser — you can check the network tab.*

Git & GitHub

Git tracks every change to the code; GitHub hosts the repository online.

IN SHRUTI · Fourteen commits, one per build step, at github.com/ProgrammarDiv/shruti.

SAY · *Every step is a commit; you can read the project's history.*

CI continuous integration

An automatic check that runs on every push: lint, type check, build.

IN SHRUTI · `.github/workflows/ci.yml` .

SAY · *Every push is lint-checked, type-checked and built automatically.*

Lint

An automatic code-quality check for mistakes and bad patterns.

IN SHRUTI · ESLint with the React rules; passes clean.

Build

Turning the source code into the optimised version that actually runs in production.

IN SHRUTI · `npm run build` .

Deployment

Vercel

Putting the app on the internet. Vercel hosts Next.js apps and redeploys on every push.

IN SHRUTI · `shruti-puce.vercel.app`.

SAY · *Pushing to main redeploys automatically.*

Production vs preview URL

Vercel gives a permanent production URL plus a temporary one per deployment. Preview URLs open only for the owner's Vercel account.

IN SHRUTI · Share `shruti-puce.vercel.app`, not the long `shruti-o1s7...` link.

HTTPS

secure origin

An encrypted connection. Browsers only allow the microphone on HTTPS or localhost.

IN SHRUTI · Voice works on the Vercel URL; not over a plain `http://192.168...` address.

SAY · *Voice needs HTTPS, which the deployment provides.*

localStorage

A small storage area inside the browser that survives refreshes.

IN SHRUTI · Demo mode keeps all data here — no server needed.

SAY · *Demo mode runs entirely in the browser; nothing leaves the device.*

Mock / demo mode

A stand-in for a real service that behaves the same way but uses fixed data.

IN SHRUTI · With no keys set, data lives in localStorage and the AI is rule-based.

SAY · *The mock is also our fallback — if the AI service fails on stage, the same screens keep working.*

Autosave

debounce

Saving automatically. “Debounce” means waiting until typing pauses so you don’t save on every keystroke.

IN SHRUTI · Saves 0.8 s after typing stops, and when you leave a field; warns before you close with unsaved text.

SAY · *A doctor never loses a sentence.*

Idempotent

Doing the same operation twice has the same effect as doing it once.

IN SHRUTI · Saves are keyed by section, so a repeated save can’t create duplicates.

SECTION 2

The database and login

The words behind “is this real?” and “is patient data safe?”

Supabase

A hosted service that gives you a Postgres database, login and file storage together.

IN SHRUTI · Database and authentication.

SAY · *Postgres and auth in one managed service – we built no login system ourselves.*

PostgreSQL

A mature, open-source relational database.

Postgres

IN SHRUTI · Seven tables.

Schema

The design of the database: which tables exist and what columns each has.

IN SHRUTI · `supabase/schema.sql`, mirroring `lib/types.ts`.

Table · row · column

A table is like a spreadsheet; a row is one record; a column is one field.

IN SHRUTI · `patients`, `consultations`, `case_sections`, `vitals`, `profiles`, `clinics`, `ai_outputs`.

Primary key

The column that uniquely identifies a row.

IN SHRUTI · `id`, a UUID, on every table.

Foreign key

A column that points to a row in another table — how tables link.

IN SHRUTI · `consultations.patient_id` → `patients.id`.

SAY · *A visit belongs to a patient by foreign key.*

UUID

A long random identifier that can't be guessed or collide.

IN SHRUTI · All IDs.

Row-level security

RLS

Rules inside the database that decide which rows each user may see or change — enforced by Postgres, not by app code.

IN SHRUTI · Every table: `clinic_id` must equal the signed-in doctor's clinic.

SAY · *Even a bug in our code can't leak another clinic's patients — the database refuses.*

Trigger

Code inside the database that runs automatically when a row is inserted or changed.

IN SHRUTI · One assigns patient codes like SHR-2026-0149; one blocks edits to a signed consultation.

SAY · *Immutability is enforced by the database, not just by hiding a button.*

Immutability

Once written, it cannot be changed.

IN SHRUTI · A signed note. Corrections would be addenda.

SAY · *Medical records are legal documents – signed notes can't be edited.*

Seed data

Realistic fake data loaded so the app isn't empty.

IN SHRUTI · `supabase/seed.sql` : Dr. Anjali Rao, eight patients, six signed visits.

SAY · *Everything you see is synthetic – no real patient data was used.*

Publishable key

anon key

The public Supabase key the browser uses. Safe to expose because RLS limits what it can do.

IN SHRUTI · `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` .

Authentication

auth

Proving who you are — logging in.

IN SHRUTI · Supabase email and password.

Session · cookie

After login the browser holds a token in a cookie so the server knows who you are on every request.

IN SHRUTI · Supabase sessions in cookies; the proxy reads them.

Password hashing

Passwords are stored as a one-way scramble that nobody — including us — can read.

IN SHRUTI · Handled by Supabase Auth.

SAY · *We never see or store passwords.*

Audit log

A record of what happened and when, that can only be added to.

IN SHRUTI · `ai_outputs` : every AI call, its input size, output, model, latency and tokens.

SAY · *You can audit every AI call the system ever made.*

SECTION 3

The AI

The words behind “how do you know it didn’t make that up?”

LLM

large language model

An AI model that reads and writes text.

IN SHRUTI · Gemini 3.6 Flash — or Claude; both are supported.

API key

A secret string that lets our server call the AI service.

IN SHRUTI · `GEMINI_API_KEY`, server-only.

Provider

The company and model behind the AI.

IN SHRUTI · Gemini or Anthropic, switchable by one setting.

SAY · *Swapping the AI provider is a config change, not a rewrite.*

Prompt

The instructions sent to the model.

IN SHRUTI · Three prompts in `lib/ai/prompts.ts`.

System prompt

The part of the prompt that sets the rules the model must follow.

IN SHRUTI · “Record only what was said. Never suggest a diagnosis. Quote your source.”

SAY · *The rules are in the prompt — and the important ones are enforced again in code.*

Structured output

JSON mode

Asking the model to answer in a fixed data format instead of free text.

IN SHRUTI · The model returns the six sections as JSON.

Schema validation

zod

Checking that the model’s answer has exactly the expected shape, and rejecting it otherwise.

IN SHRUTI · Every AI response is validated; a bad one is retried once, then falls back.

SAY · *A malformed AI answer can’t reach the screen.*

Hallucination

When an AI states something that wasn't in its input.

IN SHRUTI · The central risk in clinical documentation — and what the next three terms are for.

Source quote

The exact words from the transcript that justify each AI-written section.

IN SHRUTI · Every section carries one; open it under “Source in transcript”.

SAY · *Every AI sentence points at the words the patient actually said.*

Source-quote verification

The server checks that the quote really appears in the transcript; if not, the section is dropped and reported.

IN SHRUTI · `quoteIsInTranscript()` in `lib/ai/server.ts`.

SAY · *It isn't a prompt instruction we hope the model follows — the server drops any section whose quote isn't literally in the transcript.*

Confidence

The model's own 0–100% estimate of how well the quote supports the text.

IN SHRUTI · Shown next to each AI badge; below 60% the model leaves the field blank.

SAY · *Uncertain means blank — never guessed.*

Provenance

Where a piece of text came from.

IN SHRUTI · Four states on every section: Doctor, AI draft, AI accepted, AI edited.

SAY · *Every word in the signed record carries who wrote it.*

Accept · reject · edit

The doctor's decision on each AI-drafted section.

IN SHRUTI · Amber draft → green when accepted, cleared when rejected, blue when edited.

SAY · *Nothing AI-written enters the record until a doctor accepts it. The review page refuses to sign while a draft is undecided.*

Gap check

completeness score

The AI reads the record and lists what a complete case sheet would normally include but this one doesn't.

IN SHRUTI · The Gaps panel; re-runs after every save.

SAY · *It checks the record, not the patient – it never suggests a diagnosis or a test.*

Streaming

Receiving the AI's answer word by word instead of waiting for all of it.

IN SHRUTI · The summary streams onto the review page.

Tokens

The units an AI reads and writes in — roughly three-quarters of a word. Cost and limits are counted in tokens.

IN SHRUTI · Logged per call in `ai_outputs`.

SAY · *A consultation costs a few paise in tokens.*

Thinking level

effort

How much the model reasons before answering. More is slower and costlier.

IN SHRUTI · Low for the gap check (fast), medium for structuring and the summary.

Rate limit

The maximum calls per minute a provider allows, especially on free tiers.

IN SHRUTI · The gap check waits three seconds after a save so it doesn't fire on every pause.

Fallback

What happens when the AI call fails, times out or refuses.

IN SHRUTI · The rule-based mock answers instead, and the result is labelled “mock — fallback”.

SAY · *If the network dies on stage, the same buttons keep working.*

Refusal

When the model declines a request on safety grounds.

IN SHRUTI · Handled explicitly; falls back.

Web Speech API

The browser's built-in speech-to-text. Free, and Chrome supports Indian languages.

IN SHRUTI · The Voice tab; the language selector maps to hi-IN, ta-IN and so on.

SAY · *Zero-cost speech recognition; in production we'd swap in a medical-vocabulary model behind the same interface.*

Transcript

The text of what was said, in the original language.

IN SHRUTI · Stored with the consultation and shown next to the AI’s draft.

The two “seams”

This is the architecture story. Say it once, clearly, and the technical judge is on your side.

Interface

A contract listing what a piece of code must be able to do, without saying how.

IN SHRUTI · `DataClient` for data, `AIClient` for AI.

Seam

One place in the code where an implementation can be swapped without touching anything else.

IN SHRUTI · `lib/api/index.ts` and `lib/ai/index.ts` — one line each.

SAY · *We built the app against two interfaces. Swapping mock data for Postgres, or Gemini for Claude, is one line. Nothing in the screens changed.*

Mock client · live client

Two implementations of the same interface: fixed data, or the real service.

IN SHRUTI · Mock = localStorage and rules. Live = Supabase and Gemini.

SECTION 5

Clinical and regulatory terms

The ones a doctor or a policy person on the panel may probe.

OPD	Out-patient department — patients who visit and go home the same day. IN SHRUTI · The setting Shruti is designed for.
Case sheet case taking	The structured record a doctor writes for a consultation. IN SHRUTI · Six sections: chief complaint, HPI, past history, medications & allergies, examination, assessment & plan.
Chief complaint	The main problem in the patient's words, with its duration. IN SHRUTI · "Fever for 2 days, worse at night."
HPI	History of present illness — the story of the current problem.
Vitals	Blood pressure, pulse, temperature, breathing rate, SpO ₂ , height, weight. IN SHRUTI · Entered with units; out-of-range values flagged; impossible values rejected.
BMI	Weight divided by height squared — a weight-to-height measure. IN SHRUTI · Computed, never typed; category shown.
SpO₂	Blood oxygen level, as a percentage. IN SHRUTI · Flagged below 95%.
Carry-forward	Copying stable history from the last visit so the doctor confirms rather than retypes. IN SHRUTI · Past history and medications, badged "From last visit".
Sign & lock	The doctor attests the note and it becomes read-only. IN SHRUTI · Enforced by a database trigger.
Addendum	A dated correction added below a signed note — never an edit. IN SHRUTI · Documented as the correction path; the UI for it is future scope.

DPDP Act 2023

India's data-protection law: consent, purpose limitation, and rights over one's data.

IN SHRUTI · A consent checkbox with a timestamp at registration.

ABHA · ABDM

India's national health ID, and the mission behind it.

IN SHRUTI · An optional ABHA field on the patient; integration is on the roadmap.

NMC registration number

A doctor's licence number; it must appear on prescriptions and notes.

IN SHRUTI · Printed on every signed note.

Medical device

regulatory

Software that suggests diagnoses or treatment is regulated like a device.

IN SHRUTI · We deliberately don't — documentation only.

The three answers

q Does it diagnose?

No, by design. It structures what was said and flags what isn't documented. Diagnostic suggestions would make it a regulated medical device and would need clinical validation we haven't done.

q How do you know the AI didn't make something up?

Three layers, all on screen. Every section must quote the transcript. The server drops any section whose quote isn't literally there. And nothing is saved as the doctor's until they accept it.

q Is any of this real?

Real Postgres with row-level security, real login, real Gemini calls logged in an audit table, deployed on Vercel. The demo data is synthetic. Ask us to refresh the page — it isn't local state.